



RELAZIONE DI PROGETTO – HOMEWORK 2

Corso di Laurea: Corso di Laurea Triennale in Informatica

Anno Accademico 2025/2026

Traccia 3: Sistema Informativo per la gestione dell'orario delle lezioni

Gruppo 2:

- Girolamo Izzo – DE1000124
- Alessio Fontana – DE1000196
- Vitaliano Liguori – DE1000227

Link alla repository GitHub ufficiale del progetto: <https://github.com/OOP2026/Gruppo02>

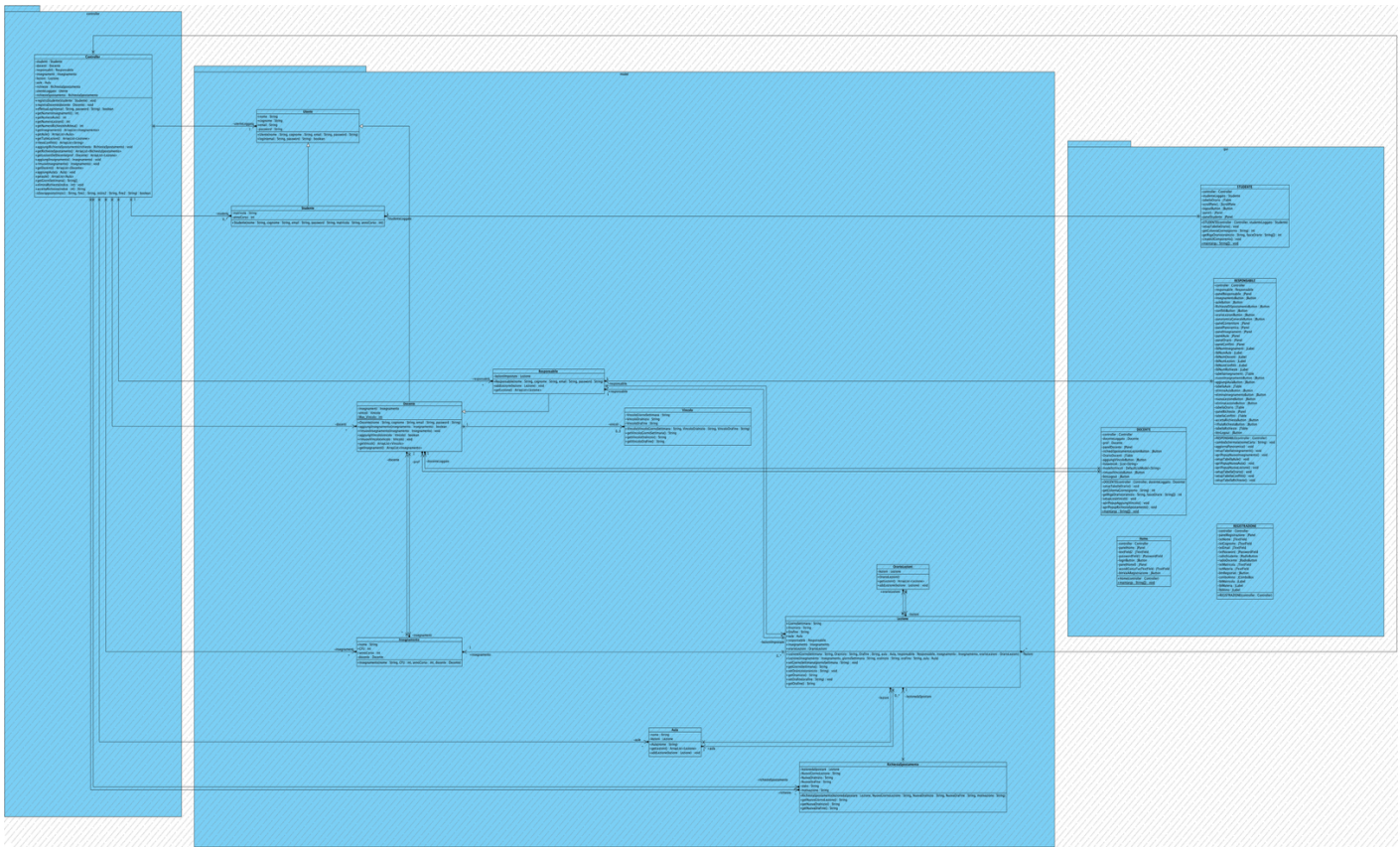
1. Obiettivo dell'Homework

Il presente documento descrive lo sviluppo e l'implementazione della seconda fase del progetto, relativo al sistema di gestione dell'orario universitario. Partendo dal package `model` sviluppato nell'Homework precedente, l'obiettivo di questa fase è stato quello di implementare l'interazione con l'utente fornendo un'interfaccia grafica (GUI) tramite la tecnologia **Java Swing UI Designer**. Per garantire un'architettura software robusta è stato introdotto il package `controller`, incaricato di fare da intermediario esclusivo tra la parte visiva (`gui`) e i dati (`model`). In questa fase, come da specifiche, non è stata gestita la persistenza dei dati su database relazionale: le informazioni sono mantenute in memoria a runtime.

2. Architettura del Sistema (Pattern MVC)

Il codice è organizzato in tre package principali:

- **model:** Contiene le classi di dominio (Utente, Studente, Docente, Responsabile, Lezione, Insegnamento, Aula, Vincolo, RichiestaSpostamento). Rappresenta lo stato del sistema e le regole base.
- **gui:** Contiene le classi grafiche estese da `JFrame`. Gestisce gli input dell'utente e mostra i dati a schermo. Non manipola mai direttamente il Model, ma delega le richieste al Controller.
- **controller:** Contiene la logica applicativa. Funge da "cervello" del sistema e da database temporaneo in memoria.



3. Analisi dei Package e delle Classi

3.1 Il Package `controller`

Il package contiene un'unica classe, `Controller.java`, che agisce come gestore centrale.

- **Gestione della Memoria:** Utilizza diverse strutture dati (`ArrayList`) per memorizzare istanze di Studenti, Docenti, Responsabili, Lezioni, Insegnamenti, Aule e Richieste di spostamento.
- **Logica di Login:** Il metodo `effettuaLogin` verifica le credenziali sulle liste degli utenti e restituisce un booleano salvando in memoria l'`utenteLoggato`.
- **Risoluzione Conflitti:** Implementa il metodo `rilevaConflitti`, il quale scansiona l'intero orario per verificare che non vi siano sovrapposizioni (due lezioni nella stessa aula alla stessa ora, o un docente impegnato in due lezioni simultaneamente).
- **Approvazione Richieste:** Il metodo `accettaRichiesta` integra una logica di validazione rigorosa: prima di spostare una lezione, verifica sia l'assenza di conflitti con altre lezioni, sia il rispetto dei vincoli di indisponibilità (massimo 3 per docente).

3.2 Il Package `gui`

Tutte le finestre richiedono l'istanza del `Controller` al momento dell'inizializzazione, passata tramite il costruttore, garantendo la condivisione dello stesso stato applicativo. Le interfacce principali sono:

- **Home:** Schermata di accesso (Login). Dopo la validazione delle credenziali da parte del `Controller`, utilizza l'operatore `instanceof` sull'`utenteLoggato` per instanziare e mostrare la dashboard corretta (Studente, Docente o Responsabile).
- **REGISTRAZIONE:** Permette l'iscrizione al sistema di nuovi Studenti e Docenti. Sfrutta i `JRadioButton` per rendere l'interfaccia dinamica: selezionando il tipo di utente, alcuni campi (es. Matricola o Materia) vengono nascosti o mostrati (`setVisible`) in tempo reale.
- **STUDENTE:** Mostra la tabella dell'orario filtrata automaticamente in base all'anno di corso e agli insegnamenti previsti per quello specifico studente.
- **DOCENTE:** Permette al professore di visualizzare il proprio orario personale, di aggiungere vincoli di indisponibilità oraria e di inviare richieste di spostamento lezione per la validazione da parte del Responsabile.
- **RESPONSABILE:** È il pannello di controllo avanzato. Tramite un sistema a schede/bottoni, permette di definire gli Insegnamenti, registrare le Aule, pianificare le Lezioni, visualizzare i Conflitti rilevati dal `Controller` ed evadere (accettare/rifiutare) le richieste di spostamento.

5. Scelte Implementative di Rilievo

- **Iniezione delle Dipendenze:** Il `Controller` viene passato metodicamente da una GUI all'altra (es. dalla `Home` alla `REGISTRAZIONE` e viceversa). Questo ci assicura che i dati inseriti non vadano persi durante la navigazione tra le finestre, simulando il comportamento di una sessione utente e di un database.
- **UX/UI Design:** Le tabelle (`JTable`) dell'orario sono state popolate dinamicamente tramite `DefaultTableModel`, posizionando le celle incrociando i metodi `getter` dei giorni della settimana e delle fasce orarie, offrendo una rappresentazione visiva chiara (simile a un classico tabellone cartaceo).

- **Gestione Errori:** Ogni azione critica dell'utente (login fallito, campi vuoti, limite massimo di 3 vincoli raggiunto) è presidiata da validazioni che restituiscono feedback visivi immediati tramite `JOptionPane.showMessageDialog()`.